

神经网络反向传播算法学习笔记

Chaoran Yang

2026 年 6 月 20 日

前言

本笔记旨在从零开始理解神经网络的核心训练算法——反向传播。全程采用分量指标形式，避免矩阵记号带来的抽象感；使用 NumPy 徒手实现全部内容，避免深度学习框架的黑箱感，便于初学者掌握细节。

1 神经网络的基本结构

我们考虑一个三层网络作为起点：输入层 ($l = 1$)，隐藏层 ($l = 2$)，输出层 ($l = 3$)，分别有神经元 N_1, N_2, N_3 个。

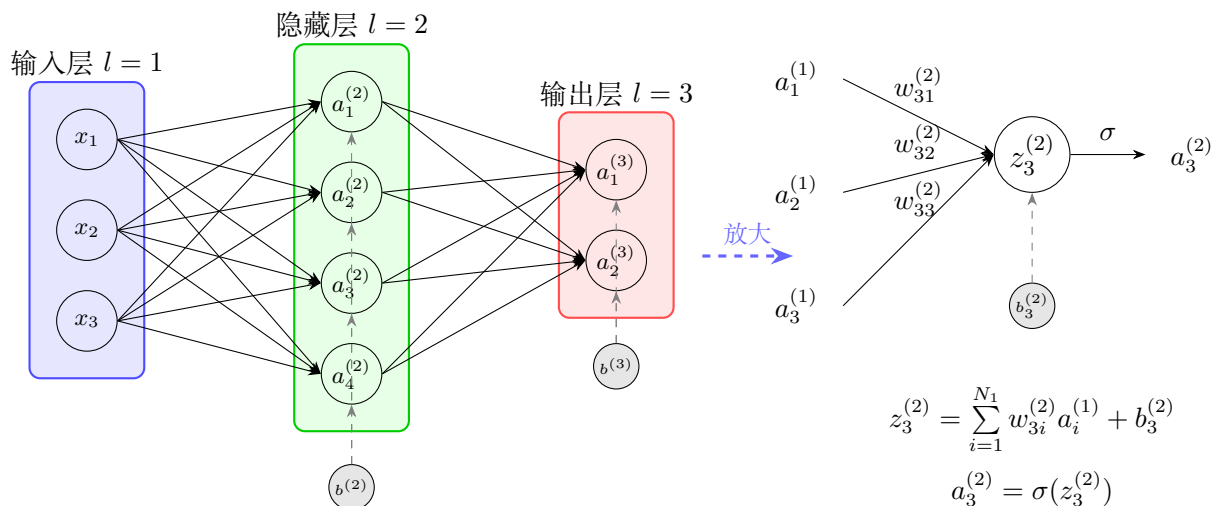


图 1: 左侧：三层神经网络；右侧：单个神经元（第 2 层第 3 个）的图示。

1.1 符号定义

- 输入： x_i 表示第 1 层第 i 个神经元的输入值（也是激活值）。
- 权重： $w_{ji}^{(2)}$ 表示从第 1 层第 i 个神经元到第 2 层第 j 个神经元的权重。上标表示连接的目标层。
- 偏置： $b_j^{(2)}$ 表示第 2 层第 j 个神经元的偏置。
- 加权输入：

$$z_j^{(2)} = \sum_{i=1}^{N_1} w_{ji}^{(2)} x_i + b_j^{(2)}$$

- 激活值： $a_j^{(2)} = \sigma(z_j^{(2)})$ ，其中 σ 是激活函数（如 Sigmoid）。

类似地，输出层定义为：

$$z_k^{(3)} = \sum_{j=1}^{N_2} w_{kj}^{(3)} a_j^{(2)} + b_k^{(3)}, \quad a_k^{(3)} = \sigma(z_k^{(3)})$$

1.2 损失函数

对于单个样本，采用平方误差损失：

$$\mathcal{L} = \frac{1}{2} \sum_{k=1}^{N_3} (y_k - a_k^{(3)})^2$$

其中 y_k 是真实标签。

2 反向传播算法推导

目标：计算损失函数 $\mathcal{L}$ 对所有权重和偏置的梯度。

2.1 输出层权重 $w_{kj}^{(3)}$

根据链式法则：

$$\frac{\partial \mathcal{L}}{\partial w_{kj}^{(3)}} = \frac{\partial \mathcal{L}}{\partial a_k^{(3)}} \cdot \frac{\partial a_k^{(3)}}{\partial z_k^{(3)}} \cdot \frac{\partial z_k^{(3)}}{\partial w_{kj}^{(3)}}$$

计算各因子：

$$\frac{\partial \mathcal{L}}{\partial a_k^{(3)}} = -(y_k - a_k^{(3)}), \quad \frac{\partial a_k^{(3)}}{\partial z_k^{(3)}} = \sigma'(z_k^{(3)}), \quad \frac{\partial z_k^{(3)}}{\partial w_{kj}^{(3)}} = a_j^{(2)}$$

因此：

$$\frac{\partial \mathcal{L}}{\partial w_{kj}^{(3)}} = -(y_k - a_k^{(3)}) \sigma'(z_k^{(3)}) a_j^{(2)}$$

定义输出层误差项：

$$\delta_k^{(3)} = \frac{\partial \mathcal{L}}{\partial z_k^{(3)}} = -(y_k - a_k^{(3)}) \sigma'(z_k^{(3)})$$

则梯度简化为：

$$\boxed{\frac{\partial \mathcal{L}}{\partial w_{kj}^{(3)}} = \delta_k^{(3)} a_j^{(2)}}$$

偏置梯度：

$$\boxed{\frac{\partial \mathcal{L}}{\partial b_k^{(3)}} = \delta_k^{(3)}}$$

2.2 隐藏层权重 $w_{ji}^{(2)}$

此时， $w_{ji}^{(2)}$ 通过影响 $z_j^{(2)}$ 进而影响所有输出层神经元。

$$\frac{\partial \mathcal{L}}{\partial w_{ji}^{(2)}} = \frac{\partial \mathcal{L}}{\partial z_j^{(2)}} \cdot \frac{\partial z_j^{(2)}}{\partial w_{ji}^{(2)}} = \delta_j^{(2)} x_i$$

其中关键在 $\delta_j^{(2)}$ ：

$$\delta_j^{(2)} = \frac{\partial \mathcal{L}}{\partial a_j^{(2)}} \cdot \frac{\partial a_j^{(2)}}{\partial z_j^{(2)}} = \left(\sum_{k=1}^{N_3} \frac{\partial \mathcal{L}}{\partial z_k^{(3)}} \cdot \frac{\partial z_k^{(3)}}{\partial a_j^{(2)}} \right) \sigma'(z_j^{(2)})$$

已知：

$$\frac{\partial \mathcal{L}}{\partial z_k^{(3)}} = \delta_k^{(3)}, \quad \frac{\partial z_k^{(3)}}{\partial a_j^{(2)}} = w_{kj}^{(3)}$$

所以：

$$\delta_j^{(2)} = \left(\sum_{k=1}^{N_3} \delta_k^{(3)} w_{kj}^{(3)} \right) \sigma'(z_j^{(2)})$$

最终隐藏层权重梯度：

$$\frac{\partial \mathcal{L}}{\partial w_{ji}^{(2)}} = \delta_j^{(2)} x_i, \quad \frac{\partial \mathcal{L}}{\partial b_j^{(2)}} = \delta_j^{(2)}$$

3 反向传播的本质：反向、传播与高效性

3.1 何为“反向”与“传播”

- **反向**：计算顺序从输出层向输入层逆向进行。先算 $\delta^{(3)}$ ，再算 $\delta^{(2)}$ ，如果继续往前则算 $\delta^{(1)}$ 。
- **传播**：指这种由一层的数据算下一层的递推运算，不论前向反向。例如 $\delta_j^{(2)} = (\sum_k \delta_k^{(3)} w_{kj}^{(3)}) \sigma'(z_j^{(2)})$ ，其中 $\sum_k \delta_k^{(3)} w_{kj}^{(3)}$ 就是将输出层误差“传播”回隐藏层。

3.2 为什么反向传播极大降低了计算成本？

对比两种方法：

- **直接法（不用反向传播）**：对每个权重单独求导，需要重复展开整个链式法则，且许多中间导数（如 $\partial \mathcal{L} / \partial z_k^{(3)}$ ）会被重复计算。总复杂度约为 $O(\text{参数数量} \times \text{层数})$ 。
- **反向传播**：先一次性算出所有 δ ，然后每个权重的梯度仅需一次乘法（ $\delta_i^{(l)} a_j^{(l-1)}$ ）。总计算量约为两次前向传播的成本，即 $O(\text{参数数量})$ 。

示例数字：假设输入 1000、隐藏 1000、输出 10。反向传播前向约 1,010,000 次运算，反向约 2 3 百万次；而直接法可能需 $1,010,000 \times 10 \approx 10,100,000$ 次以上。对于更深网络差距指数级放大。

直观理解：反向传播缓存了每层的误差项 δ ，避免了重复计算公共子表达式。

3.3 人类徒手计算

就算对于人类徒手计算，也存在反向比前向高效的问题。假设不按上面教程先推导 $\mathcal{L}$ 对第三层的梯度，而是先推导对第二层的梯度，仍会得到一样的表达式，但是我根本不知道（or 无法先验地知道，由于还没算第三层）里面求和中每个 $w^{(3)}$ 前的系数（即教程里的 $\delta_k^{(3)}$ ，但实际是一坨表达式）其实直接就能给出第三层的梯度！这导致我还得专门去算 $\mathcal{L}$ 对第三层的。结果一算发现正是刚才算过的东西，后悔了。于是知道下次推导新的网络（比如 5 层），应该先算后面的。

4 推广到任意层数网络

设网络有 L 层，第 l 层神经元个数为 N_l 。定义误差项：

$$\delta_i^{(l)} = \frac{\partial \mathcal{L}}{\partial z_i^{(l)}}$$

4.1 四个基本方程（分量形式）

4.1.1 输出层误差

$$\delta_i^{(L)} = \frac{\partial \mathcal{L}}{\partial a_i^{(L)}} \sigma'(z_i^{(L)})$$

具体形式依赖损失函数，如平方损失时 $\partial \mathcal{L} / \partial a_i^{(L)} = -(y_i - a_i^{(L)})$ 。

4.1.2 误差反向传播

$(l = L - 1, L - 2, \dots, 2):$

$$\delta_i^{(l)} = \left(\sum_{k=1}^{N_{l+1}} \delta_k^{(l+1)} w_{ki}^{(l+1)} \right) \sigma'(z_i^{(l)})$$

证明：因为 $z_i^{(l)}$ 影响下一层所有 $z_k^{(l+1)}$ ，链式法则展开：

$$\delta_i^{(l)} = \frac{\partial \mathcal{L}}{\partial z_i^{(l)}} = \sum_{k=1}^{N_{l+1}} \frac{\partial \mathcal{L}}{\partial z_k^{(l+1)}} \cdot \frac{\partial z_k^{(l+1)}}{\partial a_i^{(l)}} \cdot \frac{\partial a_i^{(l)}}{\partial z_i^{(l)}} = \sum_{k=1}^{N_{l+1}} \delta_k^{(l+1)} \cdot w_{ki}^{(l+1)} \cdot \sigma'(z_i^{(l)})$$

4.1.3 权重梯度

$$\frac{\partial \mathcal{L}}{\partial w_{ij}^{(l)}} = \delta_i^{(l)} a_j^{(l-1)}$$

因为 $z_i^{(l)} = \sum_j w_{ij}^{(l)} a_j^{(l-1)} + b_i^{(l)}$ 。

4.1.4 偏置梯度

$$\frac{\partial \mathcal{L}}{\partial b_i^{(l)}} = \delta_i^{(l)}$$

4.2 完整算法步骤

1. **前向传播**：计算并存储所有 $z_i^{(l)}, a_i^{(l)}$ 。
2. **反向传播**：
 - 计算输出层 $\delta_i^{(L)}$ 。
 - 从 $l = L - 1$ 到 2 循环，用递推式计算 $\delta_i^{(l)}$ 。
3. **计算梯度**：利用 $\delta_i^{(l)}$ 和激活值计算所有参数梯度。

5 训练过程与样本空间

5.1 多样本训练与参数共享

之前推导均针对单个样本 (x, y) 。实际训练集有 M 个样本 $(x^{(1)}, y^{(1)}), \dots, (x^{(M)}, y^{(M)})$ 。总损失定义为各样本损失的平均：

$$\mathcal{L}_{\text{total}} = \frac{1}{M} \sum_{k=1}^M \mathcal{L}^{(k)}$$

其中 $\mathcal{L}^{(k)}$ 是第 k 个样本的损失。总梯度为各样本梯度的平均值：

$$\frac{\partial \mathcal{L}_{\text{total}}}{\partial w} = \frac{1}{M} \sum_{k=1}^M \frac{\partial \mathcal{L}^{(k)}}{\partial w}$$

所有样本共用同一组参数 (w, b) ，更新时使用平均梯度。不会产生“每个样本一套参数”的情况。

5.2 梯度下降更新规则

批量梯度下降 (BGD)：

$$w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial \mathcal{L}_{\text{total}}}{\partial w}, \quad b_{\text{new}} = b_{\text{old}} - \eta \frac{\partial \mathcal{L}_{\text{total}}}{\partial b}$$

其中 η 是学习率。实际常用随机梯度下降 (SGD) 或小批量梯度下降，每次只用部分样本估计梯度，但参数仍是共享的。

5.3 样本空间的理解

- **样本空间**：所有可能的输入-输出对 (x, y) 的集合（通常来自未知分布 $P(x, y)$ ）。
- **训练集**：从样本空间中抽取的 M 个观测。
- **目的**：找到参数使在整个样本空间上的期望损失最小，但实际只能最小化训练集上的经验损失。
- 初学者可先忽略概率抽象，将样本视为固定列表即可。

6 具体例子

6.1 西瓜价格预测

学习者用西瓜价格预测来类比：输入 $\mathbf{x}$ 包含重量、含糖量、脆度、水分、产地、新鲜度 6 个特征；输出 y 是价格（一维）。神经网络第一层有 6 个输入节点（神经元），最后一层 1 个输出节点。这本质上是数据拟合未知函数 $y = f(\mathbf{x})$ 。

6.2 输出高维的现实例子

- 多标签分类（同时识别图片中的猫和狗）。
- 多变量回归（同时预测房价和面积）。
- 目标检测（输出边界框坐标和类别）。
- 图像生成（输出像素矩阵）。

6.3 曲线拟合是否属于机器学习？

是的。最简单的曲线拟合 $y = f(x)$ 输入和输出均为一维，神经网络通过训练参数来逼近真实函数，这是机器学习的基础形式。

6.4 图片分类的输入输出

- **输入**：将 $p_x \times p_y$ 像素的彩色图片展平为 $p_x \times p_y \times 3$ 维向量（RGB 三通道）。
- **输出**：对于 C 类分类任务，输出层有 C 个神经元，常用 Softmax 激活函数输出概率分布。

7 编程注意事项

Python 里可迭代对象的索引总是从 0 开始，而数学上的标号从 1 开始。如图1所示，第一层是输入层，但是每个神经元没有权重，且为恒等激活， $z_i^{(1)} = a_i^{(1)} = x_i^{(1)}$ 。于是编程时或许可以把储存所有权重的列表写成 $[W_0, W_1]$ ，其中 W_0 是第二层（隐藏层）的所有权重（表示为矩阵）。但这种写法会对编程者造成很强的干扰，首先是因为数学的标号和列表索引直接差出去 2， W_0 是所有第 $l = 2$ 层神经元的权重，却为索引 0。更麻烦的是，这种写法造成了编程语句与数学公式上标的不匹配，极易出错。前向传播的数学应为 $z_i^{(l)} = \sum_{j=1}^{N_{l-1}} w_{ij}^{(l)} a_j^{(l-1)} + b_i^{(l)}$ ，储存每层所有神经元输出值的列表为 $[z_0, z_1, z_2]$ （ z_0 已被预先赋值为 x ）。在这种写法下的代码只能为（为区分 l 和 1，层标 l 为绿色）

```
for l in range(2): z[l+1]=np.einsum('ij,j->i',W[l],a[l])+b[l+1]
```

我认为编程可读性的第一要义是：从 0 还是 1 开始索引无所谓，重要的是代码的写法要和数学公式的写法一样！即数学角标和代码索引要匹配！所以在实际编程中，我使用 `None` 占位了 W 列表的第一个元素（索引 [0]）。因为每个神经元的左边连着它的输入权重（类似突触），输入层神经元是特殊的，它没有输入权重，输出为 x ，所以对应权重为 `None` 是合理的， $[None, W_1, W_2]$ 分别为连在每层神经元的权重。但同时也要注意，循环应从索引 [1] 开始，因为 `None` 什么都没有。在这种约定下，前向传播代码可以舒服地写成：

```
for l in range(1,L): z[l]=np.einsum('ij,j->i',W[l],a[l-1])+b[l]; a[l]=sigma(z[l])
```

于是用来储存所有梯度 $\partial \mathcal{L} / \partial w$, $\partial \mathcal{L} / \partial b$ 和误差项 δ 的列表，第一个元素（索引 [0]）都用 `None` 占位，循环从 1 开始。